

業務データ共有を効率化、透明化。

BMS System

Business Management Solution **v2.0**

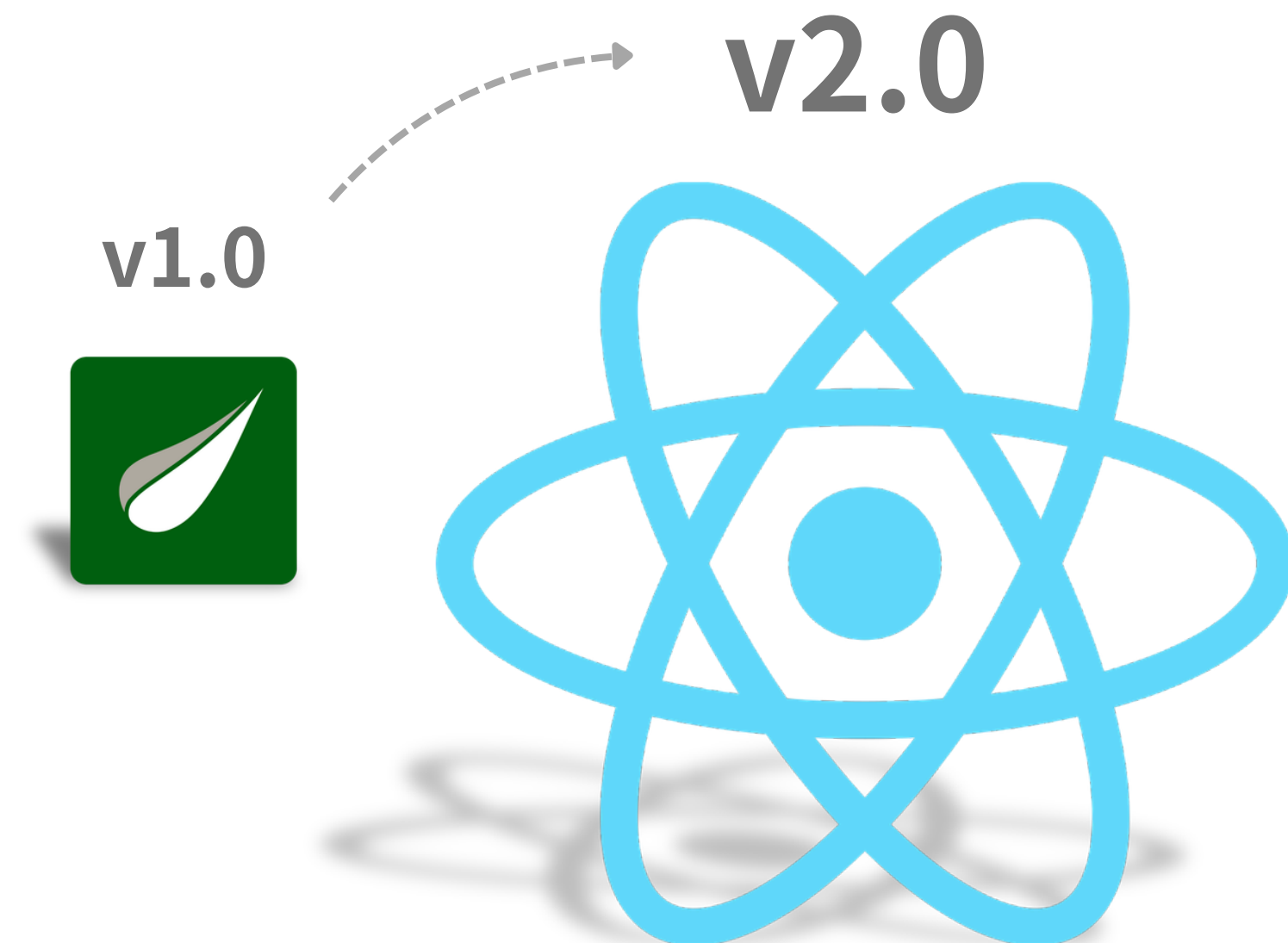
データ管理機能を1つに集約
『共有負担』をゼロにする業者間データ管理プラットフォーム

INDEX

01	制作概要		P3
02	React移行		P4
03	認証機能追加		P5
04	使用ツール		P6
05	技術構成		P7
06	システム構成		P8
07	反省点		P9

01

制作概要



1期で制作した『BMS System』を改修
フロントエンドをReact へ移行・認証機能の実装

02 React移行

① パーツの共通化（コンポーネント化）

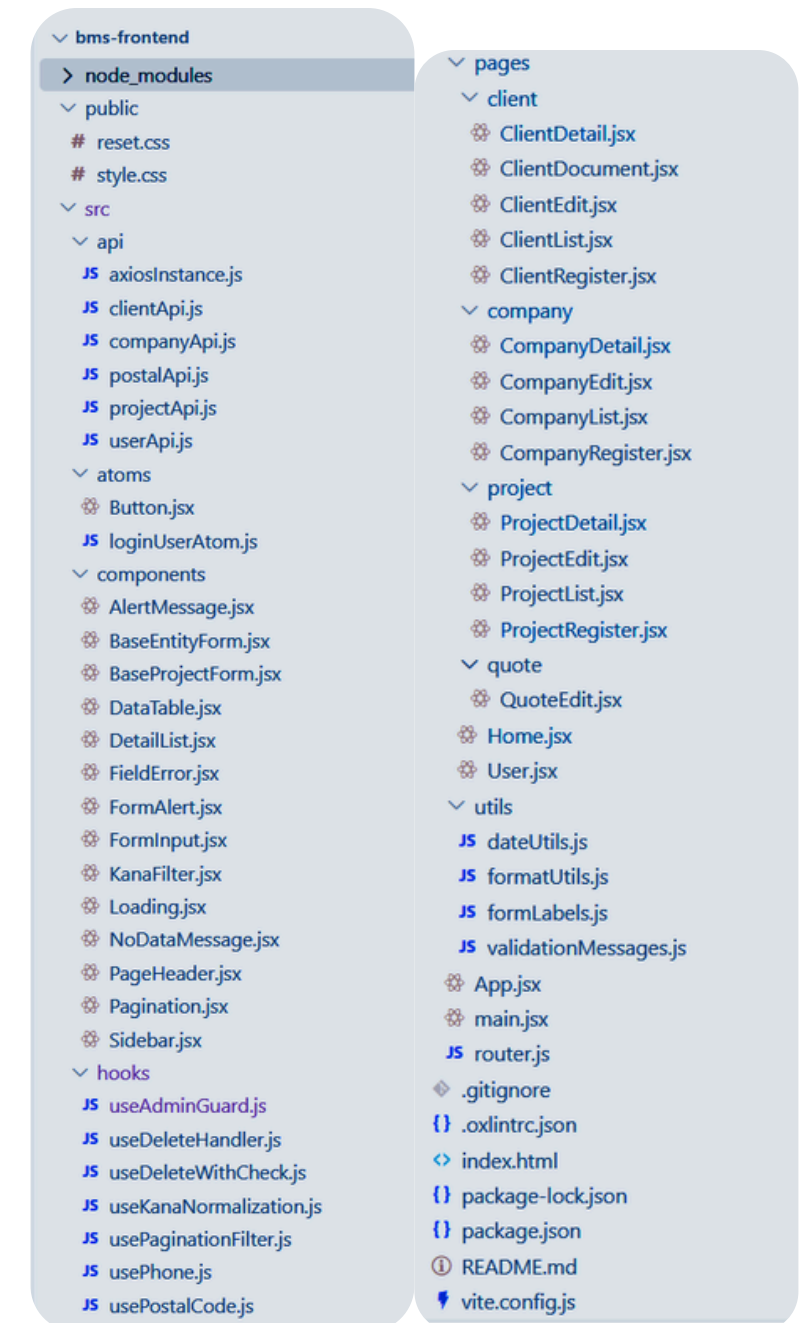
サイドバーやボタンなどの画面パーツに加え、バリデーションや入力補完などの処理も共通化し、コードの管理や再利用をしやすいにした。

② 役割ごとのファイル・フォルダ整理

画面（pages）や共通パーツ、API通信などを役割ごとにフォルダ分けし、どこに何があるか分かりやすい構成に整理した。

③ API通信の独立（バックエンドとの切り離し）

バックエンド(Spring Boot)とはAPI経由でデータをやり取りする形に改め、フロントとサーバーの役割をすっきりと分けた。



フロントエンドのモダン化と、保守性を高めた設計へ改修

03

認証機能追加(ログイン・ログアウト)

業者管理機能



新規追加機能

編集削除機能

見積追加機能

【管理者用画面】



見積判定機能

【発注業者用画面】

ユーザーの権限に応じて画面が切り替わる動的なUIを構築

04 | 使用ツール

1. 統合開発環境: Eclipse, Visual Studio Code
2. 実行環境: Node.js, Vite, Maven
3. プログラミング言語: Java, JavaScript
4. マークアップ言語・スタイルシート言語: HTML, CSS
5. サーバー (バックエンド): Apache Tomcat (Spring Boot内蔵)
6. データベース: MySQL

1. Spring Boot: Webアプリケーションの基盤（APIサーバー）
2. React: フロントエンドのUI構築（コンポーネント化）
3. MyBatis: SQLマッピング・データベース接続
4. Lombok: コードの簡素化のためのライブラリ

06

システム構成

BMS-Backend/src/main/java/com/example/app/

- └── config/ ApplicationConfig8（システムの設定）
- └── controller/ フロントからのリクエストを受け取るAPIエンドポイント
- └── domain/ ドメインモデル
- └── mapper/ データベース操作のインターフェース
- └── service/ 業務ロジック
- └── util/ FormatUtilsなどの便利ツールテンプレ
- └── ProjectSystemApplication アプリケーションのエントリーポイント

BMS-Backend/src/main/resources/

- └── mybatis/ Mybatisのマッパーxmlファイルを格納（SQL文とJavaオブジェクトを紐づけるため）
- └── application.properties アプリケーション設定ファイル（データベース接続設定やMybatis設定）
- └── validation.properties 入力チェック時のエラーメッセージ設定

bms-frontend/

- └── node_modules/
- └── public/ 静的ファイル（CSSなど）
- └── src/
 - └── api/ バックエンドと通信するAPIクライアント（Axios等）
 - └── atoms/ 小さいUIパーツ・状態管理
 - └── components/ 共通コンポーネント・フォーム部品
 - └── hooks/ カスタムフック
 - └── pages/ 各画面コンポーネント
 - └── utils/ 補助関数（フォーマット、バリデーション等）
 - └── App.jsx ルートコンポーネント
 - └── router.js （ルーティング設定）
- └── index.html HTMLテンプレート
- └── vite.config.js Viteの設定ファイル

「共通化の沼」からの脱出に苦戦

「ここも共通化できるのでは？」と欲張りすぎてしまい、どこまで作り込むべきか「やめどころ」を見失ってスケジュールが危うくなった。



今後の対応策

最初から完璧な共通化を目指さず、共通化のメリットと開発コストのバランスを見極める。



Thank you

ありがとうございました！